hashlock.

Security Audit

Nexa (Nexa Library)

October 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

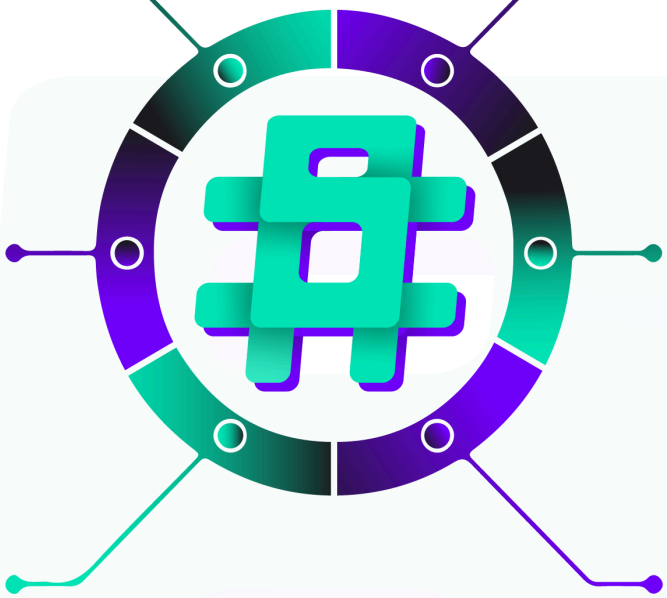
The Nexa team partnered with Hashlock to conduct a security audit of their Nexa library. Hashlock manually and proactively reviewed the code in order to ensure that the project's team and community that the deployed library is secure.

Project Context

Nexa is a decentralized cryptocurrency built on a scalable UTXO Layer-1 blockchain, aiming to address key scalability challenges in proof-of-work systems. By leveraging innovations like graphene technology and the NexScale proof-of-work algorithm, Nexa enhances transaction throughput, reduces bandwidth usage, and accelerates transaction validation. The ecosystem supports a wide range of applications, including token issuance and decentralized finance, positioning Nexa as a robust platform for high-volume, low-latency blockchain solutions. This audit report evaluates the security, compliance, and operational integrity of the Nexa blockchain infrastructure, focusing on its consensus mechanisms, and overall ecosystem resilience.

Project Name: Nexa
Project Type: Nexa Library
Compiler Version: ^5.9.2
Website: https://www.nexa.org/
Logo:

Visualised Context:



Project Name

Nexa

Launch Date

21/06/2022

Compiler Version

N/A

Language

TYPESCRIPT

JAVASCRIPT

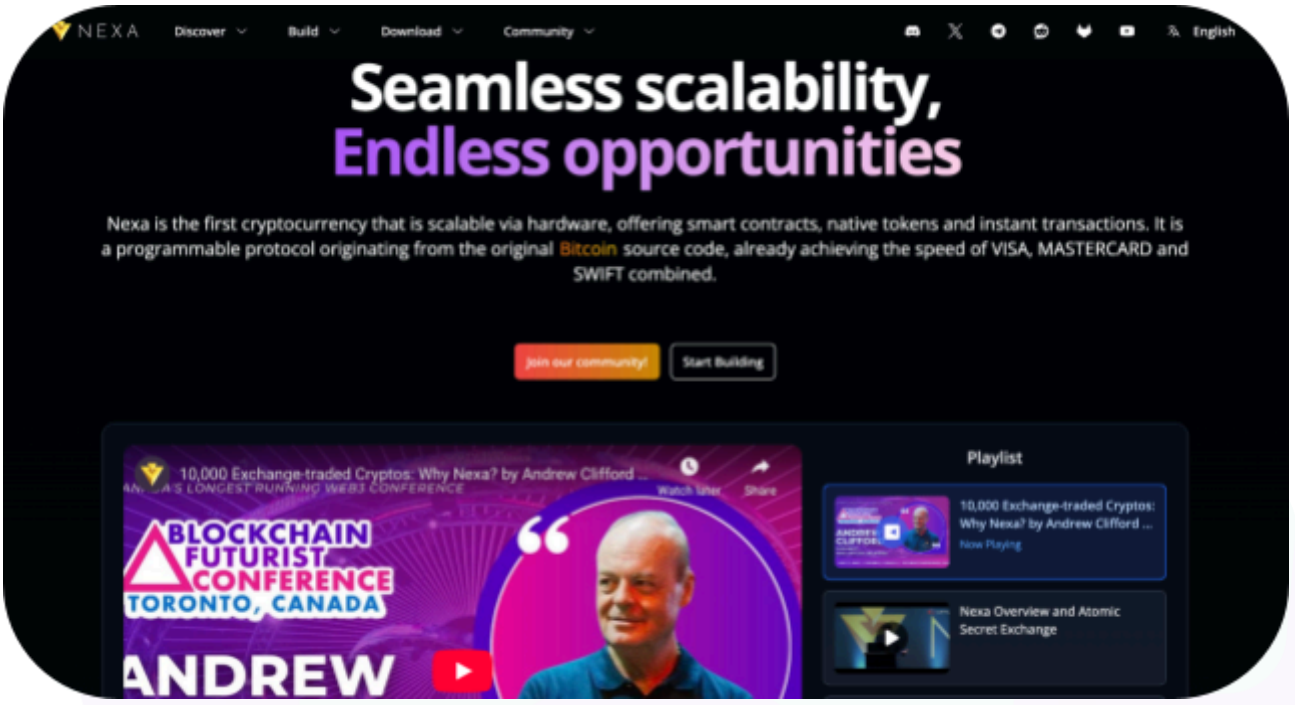
Network

NEXA

Token Ticker

N/A

Project Visuals:



Audit Scope

We at Hashlock audited the Nexa SDK code within the Nexa project. The scope of work included a comprehensive review of the Nexa SDK code listed below. We tested the SDK to check for its security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Nexa Library**
Platform	**Nexa / TypeScript / JavaScript**
Audit Date	**October, 2025**
GitLab	https://gitlab.com/nexa/libnexa-ts/-/tree/main/src
Audited GitLab Commit Hash	e27fd56bf4bb65a77e512c94237305314506 49a9
Fix Review GitLab Commit Hash	TBD

Security Rating

After Hashlock's Audit, we found the SDK to be **"Secure"**. The SDK provides a variety of low-level primitives used for interacting with Nexa.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

1 High-severity vulnerability

2 QAs

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Code Quality

This audit scope involves the Nexa SDK of the Nexa project, as outlined in the Audit Scope section. All code, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation; however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practices. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Nexa SDK code in the form of GitLab access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments and documentation are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in the Nexa SDK infrastructure are based on well-known industry standard open source projects such as bn.js, elliptic, bs58, js-big-decimal, and lodash

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] `blockheader.validProofOfWork()` - Incorrectly Calculates the Target Difficulty

Description

The `getTargetDifficulty()` helper function incorrectly determines the target difficulty when users wish to check if a PoW hash is below the current block target threshold.

Vulnerability Details

The `bitsBuf` variable is calculated using LE when it should be using BE. This causes the `exponent` and `significand` to be extremely large, rather than their desired values to produce a normal threshold value.

For example, using block 554000 with a bits value of 0x1a5daf40 should produce a difficulty of 179079 per the correctly implemented `getDifficulty()` function.

However, when getting the difficulty via `getTargetDifficulty()` for the same block, we get the following:

- `bitsBuf` = 0x1a5daf40.toBuffer(4, LE) = [0x40, 0xaf, 0x5d, 0x1a]
- `exponent` = extracts 0x40, which is 64 in decimal. (Wrong exponent!)
- `significand` = extracts [0xaf, 0x5d, 0x1a] = 11492634 (decimal)
- `target` = 11492634 * (2^(8*61) = 11492634 * (2^488) = massive target
- The target exceeds the uint256 max, meaning all PoW hashes will be incorrectly validated.

Proof of Concept

- **Location**: `blockheader.test.ts`
- **Run**: `npm run test --grep blockheader.test.ts`

```
describe('#getTargetDifficulty', () => {

  test('target difficulty failure', () => {
    let x = BlockHeader.fromObject(newbh.toObject());
    x.bits = 0x1a5daf40; // Block 554000

    let target = x.getTargetDifficulty();
    const maxPossibleHash = (2n ** 256n) - 1n;

    expect(target.toBigInt()).toBeGreaterThan(maxPossibleHash);
  });
});
```

Impact

PoW hashes are incorrectly validated, leading to unintentional upstream reverts by the node.

Recommendation

Remove `getTargetDifficulty()` and only use `getDifficulty()`.

Status

Resolved

QA

[Q-01] common.utils.cloneArray() - Fails for Nested Arrays

Vulnerability Description

Shallow copies only clone the outer array; inner arrays still have the same reference as the original.

```
// E.g.
const original = [[1, 2], [3, 4]];
const clone = cloneArray(original); // shallow copy
clone[0].push(99);
console.log(original); // [[1, 2, 99], [3, 4]] original mutated
```

Recommendation

Allow for deep cloning of arrays or rename the function to `shallowCloneArray()`.

```
public static cloneArray<T>(array: T[]): T[] {
-    return [...array];
+    return structuredClone(array); // node 17+
+    return cloneDeep(array); // or lodash func
  }
```

Status

Resolved

[Q-02] hash.hmac()– Unnecessary Custom Implementation

Vulnerability Description

The `HMAC` hash function in `hash.ts` is implemented manually. There exists an identical cryptolib version that can be used in place.

Recommendation

Consider removing the custom `hmac()` function and instead using the following functions. All tests pass in `hash.test.ts`.

```
public static sha256hmac(data: Buffer, key: Buffer): Buffer {
  return crypto.createHmac('sha256', key).update(data).digest();
}

public static sha512hmac(data: Buffer, key: Buffer): Buffer {
  return crypto.createHmac('sha512', key).update(data).digest();
}
```

Status

Resolved

Conclusion

After Hashlock's analysis, the Nexa project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the library under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the library details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the library in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the library source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this library

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

The library is deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the library can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited library.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au